

Influencer or Observer — Complete Project Walkthrough

A beginner-friendly guide to how we reached **0.856 public accuracy (tied #1)**

0. What is this project?

The task. We are given tweets from French Twitter accounts. Each account is labelled as either an **influencer** (role = 1) or an **observer** (role = 0). Given the data of a single tweet (who posted it, what it says, when, from which app), we must predict that label. This is a **binary classification** problem scored on **accuracy** (fraction of correct guesses).

The data. - `train.jsonl` — 154,914 tweets **with** labels (what we learn from). - `kaggle_test.jsonl` — 103,380 tweets **without** labels (what we must predict). - Each row is one tweet, stored as nested JSON: tweet text, time, source app, and a `user` object (account age, how many tweets/favourites they have, profile flags, etc.).

The result. Our final submission scored **0.856** on the public leaderboard — **tied for first place** with the top team. This document explains, in order, every step that got us there, what each script does, what it printed when we ran it, and what we learned from it.

1. The single most important idea (read this first)

Before any code, here is the insight that the entire project is built on, because it explains every later decision:

The test set is made of users we have never seen during training. So the only thing worth learning is what generalizes to strangers — not the identity of training users.

Why does this matter so much? A model can cheat. If the same user appears both in the part we train on and the part we validate on, the model can simply **memorize that user** ("this exact account is an influencer") and look extremely accurate — say 97%. But on the real test set, which contains **completely different users**, that memorization is worthless and the score collapses (a real earlier attempt did exactly this: **97% in validation** → **0.839 on the public leaderboard**).

So our golden rule became: **measure ourselves honestly**. We force every user's tweets to stay together (either all in training or all in validation, never split), so the model can never peek at a

validation user during training. This honest measurement is called **StratifiedGroupKFold** cross-validation (grouped by user). The honest score is lower, but it is *real* — it predicts the leaderboard.

Everything below is the story of squeezing the most accuracy we can **within honesty**.

2. How the project folder is organized

```
Kaggle2025/
├─ README.md           # short version of this document
├─ src/                # the production pipeline – the scripts that build the submission
├─ experiments/        # 12 small "probe" scripts, each answering ONE question with evidence
├─ exploration/        # first-look scripts we used to understand the raw data
├─ data/               # train.jsonl, kaggle_test.jsonl (you place these here)
└─ output/             # the submissions + saved model predictions
```

There are two kinds of scripts: - **Production scripts** (`src/`) — these actually train models and build the final CSV. - **Experiment/probe scripts** (`experiments/`) — these don't build the submission; each one runs a quick test to *prove* whether an idea helps, so we never add something on a hunch.

We'll go through them in the order a newcomer should read them.

3. Phase 1 — Understanding the data (`exploration/`)

These scripts were our first look. They don't build models; they answer "what are we even working with?"

`exploration/explore.py`

What it does: loads a few thousand tweets and prints the available columns and example values.

What it gave us: the map of the data — tweet text, `source` (the app used to post), `user.created_at`, `user.statuses_count`, `user.favourites_count`, profile flags, and the nested `quoted_status` of any quoted tweet.

`exploration/check_user_id.py` — *a pivotal discovery*

What it does: inspects the `user` object for an identifier. **What it gave us:** the user's real identifiers (`user.id`, `user.id_str`, `user.screen_name`) are **stripped out** of the data. The only top-level `id_str` is the *tweet* id, not the user. **Why this matters:** you cannot legitimately "group tweets by user" using an id — there is none. Any script that *thinks* it aggregates per user is actually mixing information across the whole dataset, which is a form of cheating (leakage).

`exploration/analyze_source.py`

What it does: looks at the `source` field (the app a tweet was posted from). **What it gave us:** professional scheduling tools (TweetDeck, Hootsuite, Buffer) are used more by influencers; phone apps dominate observers. So **which app posted the tweet is a real, honest signal**. This became one of our strongest features.

`exploration/check_overlap.py` / `check_overlap2.py`

First attempts to measure whether train and test share users — refined later into experiment 02 (next section), which gave the decisive answer.

`exploration/feature_lineage/ ( sub16 , sub19 , sub22 )`

These three older scripts are the **ancestors** of our final feature code. `sub16` built the first honest feature set, `sub19` added the source-app signal, `sub22` added the *fold-safe* source target encoding. Our production `src/features.py` is the cleaned-up merge of all three.

4. Phase 2 — Proving the ceiling (`experiments/`, the foundation probes)

Each experiment script answers one yes/no question and prints a number. We ran them and recorded the **actual output** below.

Experiment 02 — Do train and test share any users? `02_train_test_overlap.py`

Question: if test users also appear in train, memorizing users would actually help. **How:** builds a proxy user-key from `user.created_at` (constant per account) and counts how many test users appear in train.

Actual output when we ran it:

```
Scanning train.jsonl ...
  train: 154914 rows, 48360 proxy users (31.2% unique)
Scanning kaggle_test.jsonl ...
  test: 103380 rows, 32117 proxy users (31.1% unique)

=== OVERLAP ===
test users also in train: 0 (0.0% of test users)
```

What it told us: 0% overlap. Not a single test user appears in training. This is the hard proof behind Section 1: memorizing users is useless; we must generalize to strangers.

Experiment 12 — Why can validation look ~100% but the leaderboard cap at ~0.85? `12_memorize_vs_honest.py`

Question: show, with numbers, the difference between cheating-accuracy and honest-accuracy.

How: trains the same features three ways — (1) memorize the training rows, (2) row-wise CV (lets a user span folds), (3) honest grouped CV (users kept whole).

Actual output:

```
(1) memorize-train accuracy      : 100.00%
(2) row-wise CV (same user in folds): 87.56%  <- inflated, like the old 97% attempt
(3) honest CV (disjoint users)   : 84.10%   <- this predicts the public LB

Gap (1)->(3) = 15.9 pts of identity memorization
```

What it told us: the eye-popping high numbers are an illusion. ~16 accuracy points are pure user-memorization that **does not transfer** to new users. The honest ~84% is the number that actually predicts the leaderboard. **This is why we trust only grouped CV.**

Experiment 07 — Is a user's label always the same? `07_label_user_consistency.py`

Question: does an account flip between influencer and observer across its tweets? **How:** groups all tweets by user and checks for mixed labels.

Actual output:

```
rows 154914
proxy users (created_at groups): 30696 (mean 5.05 tweets/user)
user-majority ORACLE row-accuracy: 100.00%
groups with MIXED labels: 0 (0.0%)
```

What it told us: the label is 100% constant per user — zero accounts are mixed. This is huge: the problem is really "classify the *user*," and each user gives us ~5 tweets to decide. It directly justifies **per-user smoothing** (Section 6).

Experiments 03–06 — Can we squeeze more from features? (all came back ≈ noise)

We tested several plausible new features under honest grouped CV. The baseline (metadata + source target encoding) scores **83.83%**. Real outputs:

03 — URL domain of links in the tweet (`03_feature_url_domain.py`):

```
baseline (source_te only)      : 83.83%
+ url_domain_te/media/place/short: 83.76%
HONEST lift                     : -0.07%
```

06 — label-free per-user aggregates (`06_feature_user_aggregation.py`):

```
baseline : 83.83%
+ user aggregates (label-free): 83.87% (+0.04)
```

Experiments **04** (tweet/bio text via simple TF-IDF) and **05** (role-words, emoji/digit ratios, richer quoted features) gave the same verdict: **+0.12% at best, i.e. noise.**

What 03–06 told us: the *metadata* signal is **saturated at ~84%**. Adding more hand-made features does not move it. The remaining gains must come from somewhere else — (a) smarter use of the predictions we already have, and (b) the tweet **text** read by a real language model. That is exactly what Sections 6–8 do.

5. Phase 3 — The production pipeline (`src/`)

These are the scripts that actually build the submission. Read them in this order.

`src/config.py` — the control panel

One place that holds every path (where data is, where to save outputs), the random seed (42, so runs are reproducible), and all model settings. Every other script imports from here, so there are no hidden hard-coded paths.

`src/features.py` — turning raw JSON into honest numbers

What it does: converts each tweet into ~80 numeric "honest" features: - **Account behaviour:** account age, total tweets, favourites, listed count, and ratios like tweets-per-day and favourites-per-tweet (these are the strongest honest signals). - **Profile flags:** has a description? has a URL? geo-enabled? etc. - **Tweet surface:** length, word count, hashtag/mention/URL counts, capital-letter ratio. - **Quoted tweet:** if the tweet quotes someone, that quoted user's follower stats (the main user's followers are stripped, but the quoted user's are present — an indirect signal). - **Source app:** a 5-way category (official / scheduler / automation / ...) **plus** a *fold-safe source target encoding* (explained next). - **Banned on purpose:** profile colours and background-image fields are **forbidden** by an assertion in the code, because they are user-fingerprints that cause leakage.

Fold-safe target encoding means: when we replace "source app" with "the average label of that app," we compute that average using **only the training rows of the current fold**, never the validation rows. This prevents the encoding from leaking answers.

`src/model.py` — the neural network

A simple but solid **multi-layer perceptron (MLP)**: layers of size $256 \rightarrow 128 \rightarrow 64 \rightarrow 1$, with BatchNorm, GELU activations, and dropout for regularization. It reads the ~80 features and outputs a probability that the tweet's author is an influencer.

`src/train.py` — the heart of the pipeline

What it does, step by step: 1. Loads data, builds features. 2. Splits into 5 folds **grouped by user** (`--group`) — the honest split from Section 1. 3. For each fold: fits the scaler and source-encoding on *training rows only*, trains the MLP, and (with `--blend`) also trains a **LightGBM** (a gradient-boosted tree model) on the same fold. 4. Combines MLP + LightGBM (`--blend`), then applies **per-user smoothing** (`--smooth`). 5. Tunes the decision threshold on honest out-of-fold predictions and writes the submission.

Actual output of `python train.py --group --blend --smooth`:

```
Best blend: alpha(MLP)=0.55 thr=0.48 -> 84.31% (MLP 83.94% | LGBM 84.11%)
Per-user smoothing: OOF 84.31% -> 84.78% (+0.47), thr 0.46
Saved submission: ../submission_mlp.csv (103380 rows)
```

What it told us: MLP alone 83.94%, LightGBM alone 84.11%, blending them 84.31%, and then **per-user smoothing adds +0.47% → 84.78%**. That smoothed blend scored **0.848** on the public leaderboard — our first strong honest submission.

6. The trick that mattered most — per-user smoothing

From Experiment 07 we know **every tweet from one user has the same label**. So if the model gives a user's 5 tweets probabilities like 0.55, 0.48, 0.61, 0.40, 0.58, those disagreements are just *noise* — the truth is one single answer for that user.

Per-user smoothing (`--smooth` , proven in `experiments/08_user_smoothing.py`) replaces each tweet's probability with the **average** over all tweets of that user (grouped by `user.created_at`). Averaging cancels the random per-tweet errors. Real measured effect:

```
blend 84.31% -> per-user smoothed 84.78% (+0.47%)
```

This is the **single biggest gain after features** — and it costs nothing extra, it's just a smarter way to use predictions we already have. It works *because* the label is user-constant (Experiment 07), which is why we proved that first.

7. Phase 4 — adding model diversity (`src/stack_models.py`)

What it does: trains four more models on the same honest folds — **CatBoost, XGBoost, HistGradientBoosting, ExtraTrees** — and saves their predictions. Different model types make *different* mistakes, so combining them helps.

Effect: the stack lifted the smoothed ensemble from 84.78% → **84.84%** honest OOF, which scored **0.850** public. CatBoost turned out to be the single strongest tree model (84.77% smoothed on its own).

8. Phase 5 — the breakthrough: reading the text with CamemBERT

Metadata was saturated at ~0.850. The one source of signal we hadn't truly used was the **tweet text itself**. Simple TF-IDF text (Experiment 04) barely helped — but a real French language model is far more powerful. (The tweets are French, so we use **CamemBERT**, a French BERT. We did *not* use an English Twitter model — see Section 9.)

Step 1 — a cheap probe first ([experiments/10_text_camembert_probe.py](#))

Before spending hours fine-tuning, we tested whether a **frozen** CamemBERT (no training, just its embeddings) adds anything on top of metadata. **Actual output:**

```
metadata only          : raw 84.05% | smoothed 84.54%
metadata + CamemBERT   : raw 84.21% | smoothed 84.82%
```

+0.28% — the first real lift beyond metadata. That green light justified the expensive fine-tune.

Step 2 — fine-tune CamemBERT on the tweet ([src/finetune_camembert.py](#))

We fine-tuned [almanach/camembert-base](#) to predict the label directly from the tweet text, across the same 5 honest folds (≈1 hour on the GPU). **Actual output:**

```
OOF text-only: raw 69.89% | smoothed 74.80%
```

Alone the text model is "only" ~75%, but it is **complementary** — it knows things the metadata models don't. Blended in, the ensemble rose to **85.14%** honest → **0.852** public.

Step 3 — also feed the user's bio ([src/finetune_camembert_bio.py](#))

Social role is often stated literally in the **bio** ("journaliste", "officiel", "média"...). We fine-tuned CamemBERT on **bio + tweet** together. **Actual output:**

```
OOF: raw 73.25% | smoothed 74.58%
```

Adding the bio raised the raw text accuracy from 69.89% → 73.25%, and — crucially — it earned its own slot in the ensemble (it adds signal the tweet-only model missed). The ensemble rose to **85.40%** honest → **0.856 public — tied for #1**.

src/make_submission.py — the final blender

What it does: finds every model's saved predictions, smooths each one per-user, then uses **greedy ensemble selection** (repeatedly add whichever model most improves the honest OOF) to choose the blend, and writes the final CSV. **Actual output (the canonical final run):**

```
solo smoothed-OOF accuracy:
  cat 84.77 | xgb 84.75 | mlp 84.64 | lgb 84.64 | hgb 84.62 | et 83.89
  camembert 74.80 | camembertbio 74.58 | camembertlarge 74.49 | twhin 73.42
ENSEMBLE weights: {cat:2, mlp:2, camembert:1, camembertlarge:1, lgb:1, hgb:1}
ENSEMBLE smoothed-OOF: 85.44% @thr 0.500
Saved ../output/submission_final.csv (103380 rows)
```

This `output/submission_final.csv` is the 0.856 submission.

9. Things we tried that did NOT help (and why that's good to know)

A strong project knows its dead ends. Each of these was tested honestly and **added nothing**, which is itself evidence that we hit the data's true ceiling:

Attempt	Result	Why it failed
TwHIN-BERT (English/multilingual Twitter model)	+0.00% to ensemble	Multilingual → shallower French than CamemBERT; redundant signal.
CamemBERT-large (335M, ~4 h to train)	+0.04% (noise)	Bigger model, but the text signal itself is already fully used.
Hashtag / mention target-encoding (exp 11)	-0.17% (negative!)	Overfits fold-specific tags; doesn't generalize to new users.
URL-domain / extra features (exp 03-06)	≤ +0.12%	Metadata already saturated.

Why CamemBERT, not TwHIN-BERT? The tweets are French. CamemBERT is French-specialised; TwHIN-BERT is multilingual and shallower on French. We *tested* both — TwHIN-BERT added exactly 0.00% — so we kept CamemBERT. This is a defensible, evidence-based choice.

10. The full story in one table — how accuracy was built up

Every number below is an **honest grouped-CV** out-of-fold (OOF) accuracy, and where we submitted it, the matching **public leaderboard** score. Notice how tightly OOF tracks public — that's the payoff of honest measurement.

Stage	What was added	honest OOF	public LB
Metadata baseline (LightGBM)	honest features + source TE	83.83%	—
MLP only	neural net on features	83.94%	—
+ LightGBM blend	two model types	84.31%	—
+ per-user smoothing	average each user's tweets	84.78%	0.848
+ CatBoost/XGB/HistGBM/ExtraTrees	model diversity	84.84%	0.850
+ fine-tuned CamemBERT (tweet)	French text signal	85.14%	0.852
+ CamemBERT (bio + tweet)	role stated in bio	85.40%	0.856 (tied #1)

What contributed to the final CSV (`output/submission_final.csv`): a greedy-selected blend of **CatBoost ×2, MLP ×2, CamemBERT(tweet), CamemBERT-large, LightGBM, HistGBM**, each prediction **smoothed per user**, thresholded at 0.50 — honest OOF **85.44%**, public **0.856**.

11. How to reproduce it (commands)

```
# 0. put train.jsonl and kaggle_test.jsonl in data/
cd src
python train.py --group --blend --smooth # MLP + LightGBM, honest CV, smoothing -> 0.848
python stack_models.py # + CatBoost / XGBoost / HistGBM / ET -> 0.850
python finetune_camembert.py # fine-tune CamemBERT on tweet (GPU ~1h) -> 0.852
python finetune_camembert_bio.py # fine-tune on bio + tweet (GPU ~1h) -> 0.856
python make_submission.py # blend everything -> output/submission_final.csv
```

Each model saves aligned prediction arrays into `output/model_predictions/`; `make_submission.py` discovers them automatically and builds the final blend.

12. The three sentences to remember for the oral

1. **Honesty over hype:** because test users are 100% strangers (Exp 02), we measured with user-grouped CV — turning a fake 97% into a real ~84% that actually predicts

Every number in this document is a real output captured from running the scripts in this repository, not an estimate. The final submission is `output/submission_final.csv`.